

Lab 2: Reading a series, and the bar it sets

MATH840 | Time Series | Week 2 — graded assignment 1 of 7

AUTHOR

Ihor Miroshnychenko

AFFILIATION

Kyiv School of Economics

 Open the notebook

 Open in Colab

Open In Colab

Click **Copy to Drive** before you start working, otherwise nothing you write is saved.

 **This lab is on last week's lecture, not today's**

From now on the practice session runs one week behind the lecture. Everything you need today was covered in [Week 1](#) and has been published since — so missing a lecture no longer costs you the lab that follows it, and you have a week to prepare rather than ninety minutes.

Today's lecture on decomposition is next week's lab.

The task

You have a series. Today you look at it properly, commit to a claim about it, and then find out what the claim was worth.

The claim: of the four simple methods from Week 1 — mean, naive, drift, seasonal naive — **which will be hardest to beat on your series?** You answer from the graphics, before you run anything. Then you run all four and see.

 **This one counts**

8 points, produced and submitted **within the session**. Deadline is the end of the practice slot. Work is marked afterwards and returned before the next practice session.

What you are marked on

Section	Criteria	Points
1	Data preparation	1
2	EDA and visualisation	4
3	Implementation	2

Section	Criteria	Points
4	Code quality and reproducibility	1
Total		8

The notebook has one section per row. **Keep those headings** — they are how the work is marked.

Your series

The same one you were assigned in Week 1: the same student identifier produces the same dataset. If you have lost your notebook, the assignment function is in the template again.

Warning

Do not switch datasets. This series stays yours through Week 3, and next week you explain today's result on it.

Suggested pacing (80 minutes)

Minutes	What
0–10	Setup, load your series, repair the calendar
10–35	Time plot, seasonal plot, subseries plot, ACF — with readings
35–45	Commit to your prediction, in writing, before running anything
45–65	Implement the four methods, hold out the last cycle, score them
65–75	Compare against your prediction, and account for the difference
75–80	Export and submit

If you are behind at minute 75, **submit what you have**. A valid partial submission scores; a perfect missing one does not.

Section 1 — Data preparation (1 point)

Load your series and put it on a regular time grid.

- Resolve duplicated timestamps if there are any, and say in one sentence what you decided and why.
- Fill the calendar so gaps become visible missing values rather than absent rows.
- State the frequency, the span, and how many complete seasonal cycles you have.

You did all of this in Week 1. It should take ten minutes.

⚠️ ⚠️ If your series ends with missing values

Some series in the pool stop being reported years before their last row: the file ends with a long run of blanks. `interpolate()` will happily fill those by repeating the last value, and then your holdout is a flat line you invented — every method “predicts” it perfectly and the exercise is meaningless.

Trim to the last real observation before you split, and say in your report that you did and how much you dropped. Filling interior gaps is repair; filling the tail is fabrication.

⚠️ ⚠️ Where this goes wrong: the frequency string

- `"W"` does not mean “weekly”. It means *week ending Sunday*. If your timestamps are Mondays, a grid built with `"W"` lines up with nothing and your series comes back empty. Use the anchored form, `"W-MON"` — which is what `spec["freq"]` already gives you.
- pandas 3 removed the bare `"M"`, `"Q"`, `"Y"` and `"H"` aliases. Use `"MS"` / `"ME"`, `"QS"` / `"QE"`, `"YS"` / `"YE"` and lowercase `"h"`. A bare `"Q"` now raises rather than warns.
- `"MS"` is month **start**, `"ME"` is month **end**. Pick the one your timestamps already use, or every row misses the grid by a few days.

After building a grid, check you did not lose everything: `s["y"].notna().sum()` should be close to the number of rows you started with.

[offset aliases](#) · [anchored offsets](#) · [date_range](#) · [reindex](#) · [duplicated](#)

Section 2 — EDA and visualisation (4 points)

Four plots, each with a sentence saying what you see. A plot with no reading attached earns nothing.

Graphics. Time plot, seasonal plot, seasonal subseries plot, ACF. Name the patterns: trend, seasonality, cycles, level shifts, outliers. Say whether the seasonal swing grows with the level of the series — you will need that observation next week.

⚠️ ⚠️ Where this goes wrong

- `plot_acf` raises once `lags` reaches the length of the series. `lags=min(40, len(y) // 3)` is a safe default; for weekly data use at least 60 so the annual spike is visible.
- Plot the ACF with missing values removed — `plot_acf` will not handle `NaN` for you.

[plot_acf](#)

Then commit. Which of the four methods will be hardest to beat, and why? Write three or four sentences citing what is *in the plots* — the size of the seasonal swing against the size of the trend, whether the pattern repeats in the same shape, whether the level moved recently.

 Important

Write this **before** you run Section 3. The point is not the guess; it is whether you can read a series. A well-argued wrong prediction earns full marks — an unargued right one earns nothing.

Section 3 — Implementation (2 points)

The four methods. Implement them yourself — they are four lines each, and they are on the [Week 1 slides](#) if you need them.

$$\hat{y}_{T+h} = \bar{y} \quad \hat{y}_{T+h} = y_T \quad \hat{y}_{T+h} = y_{T+h-m} \quad \hat{y}_{T+h} = y_T + h \frac{y_T - y_1}{T - 1}$$

The holdout. Cut off the last full seasonal cycle — one year of data — fit nothing, and forecast that span with all four. Report the RMSE of each, and the ratio of each to the best. That ratio is the skill score from the lecture: it says how much worse a method is than the bar.

The verdict. Which won? Did your prediction hold? If it did not, say what in the plots you misread — that sentence is worth more than the prediction was.

 Where this goes wrong

- `snaive` needs the last `m` observations of the **training** part, not of the whole series. If your forecast looks suspiciously perfect, you are forecasting with the answer.
- Drop or interpolate missing values before scoring: an `NaN` anywhere in the holdout makes every RMSE `NaN`.
- RMSE is in the units of your series, so it is only comparable *between methods on the same series* — never between your series and a classmate's. That is what the ratio is for.

Section 4 — Code quality and reproducibility (1 point)

- The notebook runs top to bottom on a fresh kernel without errors.
- No dead cells, no leftover experiments.
- Every number in your text comes from a cell, not from your memory of an earlier run.

Submission

Upload three files to the Week 2 activity on Moodle **before the end of the session**:

- `SURNAME_lab02.pdf` — the rendered report:

- `SURNAME_lab02.pdf` — the rendered report;
- `SURNAME_lab02.ipynb` — the source;
- `SURNAME_lab02_summary.csv` — written by the last cell of the template.

For next week

Today's lecture covers decomposition, and next week's lab is built on it: you will take **this same series**, decompose it, and explain the result you got today — why that method won, in terms of the components of your series.

Read Chapter 3 of [FPP, the Pythonic Way](#) before then.